



DOCUMENTACIÓN TÉCNICA - XUXEMONS

1. Descripción General del Proyecto

XUXEMONS es una aplicación web full-stack basada en coleccionar, entrenar y combatir criaturas virtuales llamadas "Xuxemons". Está desarrollada con un frontend en Angular 20, un backend en Laravel 11, una base de datos MySQL 8.0, y está completamente containerizada con Docker.

2. Objetivos del Proyecto

- Proporcionar una plataforma interactiva para capturar, entrenar y evolucionar Xuxemons.
- Implementar un sistema de autenticación seguro basado en tokens (Laravel Sanctum).
- Ofrecer operaciones CRUD sobre colecciones, mochilas e inventarios.
- Permitir que los usuarios gestionen su perfil y datos personales.
- Facilitar el despliegue mediante Docker Compose en diferentes entornos.

3. Arquitectura del Frontend (Angular 20.3.0)

3.1 Estructura de Componentes

- **LoginComponent**: Gestiona el inicio de sesión de usuarios
- **RegisterComponent**: Gestiona el registro de nuevos usuarios
- **PaginaPrincipal**: Dashboard principal con información general
- **Xuxedex**: Enciclopedia o Pokédex de todas las criaturas disponibles
- **MochilaComponent**: Gestión del inventario de items y Xuxemons capturados
- **PagInfoUsuario**: Perfil del usuario con información personal

3.2 Servicios Principales

- **AuthService**: Maneja autenticación, tokens y sesiones
- **XuxemonsService**: API para obtener y manipular Xuxemons
- **MochilaService**: API para gestionar mochilas e inventarios
- **InterceptorService**: Intercepta peticiones para añadir tokens Bearer

3.3 Autenticación y Seguridad

- Los tokens de autenticación se almacenan en `localStorage`
- El `authInterceptor` añade el header `Authorization: Bearer <token>` a todas las peticiones - El `authGuard` protege las rutas que requieren autenticación
- Laravel Sanctum valida los tokens en el backend

3.4 Tecnologías del Frontend

- **Angular**: 20.3.0
- **TypeScript**: 5.9.2
- **RxJS**: 7.8.0
- **SCSS**: para estilos
- **Node**: 20 (en Docker)

4. Arquitectura del Backend (Laravel 11)

4.1 Controladores Principales

- **AuthController**: Gestiona registro, login, logout y validación de tokens
- **ColeccionController**: CRUD de colecciones de Xuxemons del usuario
- **MochilaController**: CRUD de mochilas e items
- **UserController**: Gestión de datos de usuarios

4.2 Modelos y Relaciones

- **User**: Modelo de usuario con roles (admin, jugador)
- **Xuxemon**: Criatura con propiedades (tipo, nivel, experiencia)
- **Item**: Items del juego (potions, berries, etc.)
- **Mochila**: Contenedor de items del usuario
- **Coleccion**: Registro de Xuxemons capturados por usuario
- **Enfermedad**: Estados negativos que pueden afectar Xuxemons
- **Vacuna**: Items curativos contra enfermedades
- **Relaciones**:
 - User (1) -> (inf) Coleccion
 - Coleccion (inf) -> (1) Xuxemon
 - User (1) -> (inf) Mochila
 - Mochila (1) -> (inf) Item
 - Xuxemon (inf) <- -> (inf) Enfermedad (muchos a muchos)

4.3 Rutas API

Las rutas están definidas en `routes/api.php` y protegidas por autenticación:

- `POST /register` - Registro de nuevo usuario
- `POST /login` - Inicio de sesión
- `POST /logout` - Cierre de sesión
- `GET /xuxemons` - Listar todas las criaturas
- `GET /xuxemons/{id}` - Obtener detalle de una criatura
- `POST /colecciones` - Capturar nuevo Xuxemon
- `GET /colecciones` - Listar Xuxemons del usuario
- `PUT /colecciones/{id}` - Actualizar Xuxemon (evolución, curación, etc.)
- `DELETE /colecciones/{id}` - Liberar un Xuxemon
- `GET /mochilas` - Listar items en mochila
- `POST /mochilas` - Añadir item a mochila
- `DELETE /mochilas/{id}` - Usar o descartar item

4.4 Tecnologías del Backend

- **Laravel**: 11
- **PHP**: 8.3
- **Sanctum**: autenticación basada en tokens

- **Composer**: gestor de dependencias
- **Apache**: servidor web

5. Base de Datos (MySQL 8.0)

5.1 Tablas Principales

1. users - Registro de usuarios
 - id, name, email, password, role, created_at, updated_at
2. xuxemons - Catálogo de criaturas disponibles
 - id, nombre, tipo, ataque, defensa, hp, velocidad, imagen, created_at, updated_at
3. items - Catálogo de items
 - id, nombre, tipo, valor, descripcion, created_at, updated_at
4. colecciones - Xuxemons capturados por usuarios
 - id, user_id, xuxemon_id, nivel, experiencia, created_at, updated_at
5. mochilas - Items del usuario
 - id, user_id, item_id, cantidad, created_at, updated_at
6. enfermedades - Estados negativos
 - id, nombre, descripcion, created_at, updated_at
7. vacunas - Items curativos
 - id, enfermedad_id, item_id, created_at, updated_at
8. xuxemonenfermedad - Relación de enfermedades de Xuxemons
 - id, xuxemon_id, enfermedad_id, created_at, updated_at
9. personalaccesstokens - Tokens de Sanctum
 - id, tokenable_type, tokenable_id, name, token, abilities, created_at, updated_at

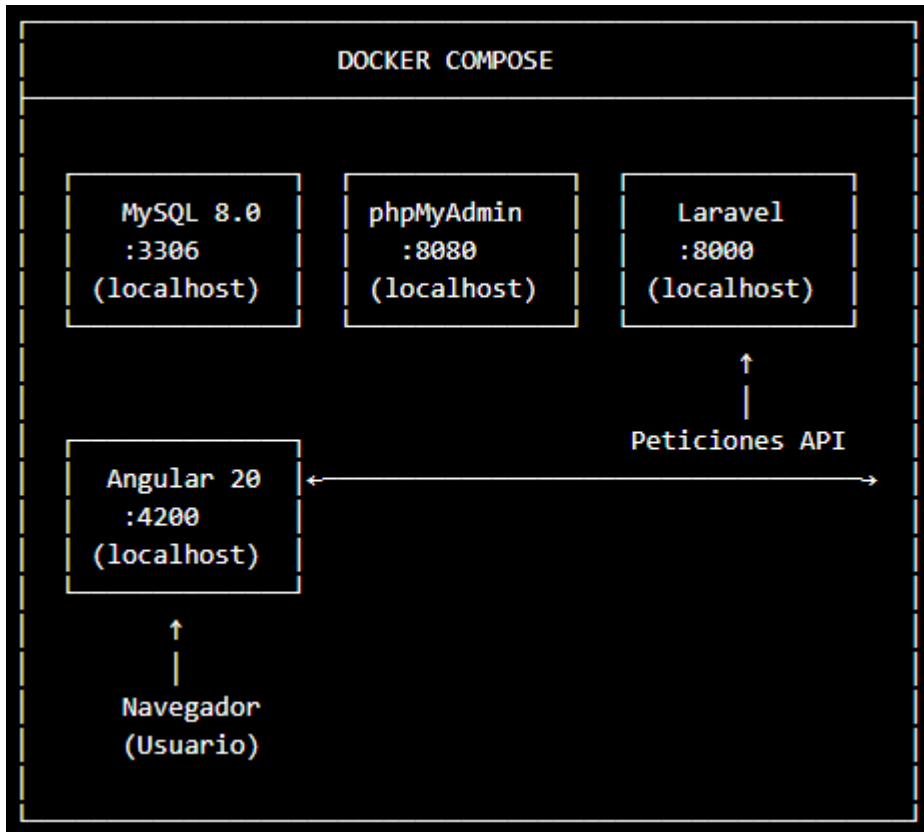
6. Arquitectura de Docker

6.1 Descripción General

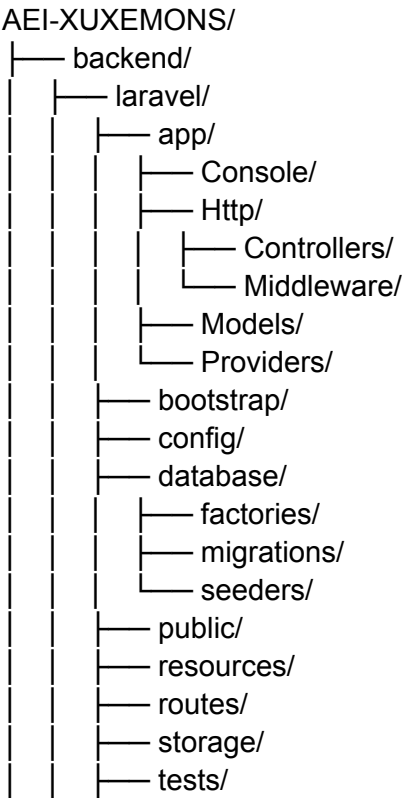
El proyecto utiliza Docker Compose para orquestar 4 servicios:

- **MySQL 8.0** (db): Base de datos
- **phpMyAdmin**: Interfaz gráfica para MySQL
- **Laravel con Apache**: API Backend
- **Angular con Node.js**: Frontend

6.2 Diagrama de Servicios



7. Estructura de Carpetas



```

├── vendor/
├── composer.json
├── package.json
├── .env.example
├── .env.docker
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── login/
│   │   │   ├── register/
│   │   │   ├── pagina-principal/
│   │   │   ├── xuxedex/
│   │   │   ├── mochila/
│   │   │   ├── pag-info-usuario/
│   │   │   ├── services/
│   │   │   ├── guards/
│   │   │   ├── interceptors/
│   │   │   └── app.routes.ts
│   │   ├── assets/
│   │   └── index.html
│   ├── package.json
│   ├── angular.json
│   └── tsconfig.json
├── docker/
│   ├── backend/
│   │   └── laravel/
│   │       └── Dockerfile
│   └── frontend/
├── mysql/
│   └── init.sql
├── docker-compose.yml
└── README.md

```

8. Tecnologías y Justificación

Tecnología	Versión	Razón
Angular	20.3.0	Framework moderno y reactivo para SPA
TypeScript	5.9.2	Type safety y mejor mantenibilidad
RxJS	7.8.0	Manejo de async y streams reactivos
Laravel	11	Framework maduro, documentado y seguro
PHP	8.3	Lenguaje estándar para web backends
Sanctum	Latest	Autenticación ligera basada en tokens
MySQL	8.0	RDBMS confiable y ampliamente soportado
Docker	Latest	Containerización para reproducibilidad
Apache	2.4	Servidor web probado y confiable
Node.js	20	Runtime de JavaScript para Angular

9. Requisitos de Despliegue

- Docker Desktop (versión 2.40.3 o superior)
- Docker Compose (incluido en Docker Desktop)
- 4 GB RAM mínimo disponible
- Puertos disponibles: 3306, 8000, 4200, 8080
- Conexión a Internet (para descargar imágenes)
- 2 GB espacio libre en disco

10. Instalación Local

10.1 Pasos previos

1. Clonar el repositorio
2. Navegar al directorio del proyecto
3. Asegurar que Docker Desktop está ejecutándose

10.2 Despliegue con Docker Compose

`docker compose up --build -d`

Este comando:

- Construye las imágenes
- Inicia todos los servicios en background
- Ejecuta migraciones y seeders automáticamente

10.3 Primera vez - Acceso

- Frontend: ``http://localhost:4200``
- Backend: ``http://localhost:8000``
- phpMyAdmin: ``http://localhost:8080``
- Usuario demo: email@ejemplo.com /

password123 **10.4 Detener servicios**

`docker compose down`

11. Variables de Entorno

11.1 .env.docker (Backend)

APPNAME=Xuxemons
APPENV=docker
APPDEBUG=true
APPURL=http://localhost:8000

DBCONNECTION=mysql

DBHOST=db
DBPORT=3306
DBDATABASE=bd-xuxemons
DBUSERNAME=xuxemons
DBPASSWORD=xuxemons

MAILFROMADDRESS=noreply@xuxemons.test

SANCTUMSTATEFULDOMAINS=localhost:4200

11.2 .env.local (para desarrollo sin Docker)

APPNAME=Xuxemons
APPENV=local
APPDEBUG=true
APPURL=http://localhost:8000

DBCONNECTION=mysql
DBHOST=127.0.0.1
DBPORT=3306
DBDATABASE=bd-xuxemons
DBUSERNAME=root
DBPASSWORD=root

MAILFROMADDRESS=noreply@xuxemons.test

SANCTUMSTATEFULDOMAINS=localhost:4200

12. Explicación del docker-compose.yml

El archivo `docker-compose.yml` define todos los servicios necesarios para ejecutar la aplicación en contenedores Docker. A continuación se detallan los servicios principales:

12.1 Servicio `db`

- **Imagen:** `mysql:8.0`
- **Nombre del contenedor:** `xuxemons-db`
- **Reinicio:** `always` (se reinicia automáticamente si falla)
- **Variables de entorno:**
 - `MYSQL\_ROOT\_PASSWORD`: contraseña root (`root`)
 - `MYSQL\_DATABASE`: base de datos inicial (`bd-xuxemons`)
 - `MYSQL\_USER`: usuario de acceso (`xuxemons`)
 - `MYSQL\_PASSWORD`: contraseña del usuario (`xuxemons`)
- **Puertos:** `3306:3306` (MySQL está disponible en `localhost:3306`)
- **Volúmenes:** `db\_data:/var/lib/mysql` (persiste datos entre reinicios)
- **Healthcheck:** verifica que MySQL responda cada 10 segundos

12.2 Servicio `phpmyadmin`

- **Imagen:** `phpmyadmin/phpmyadmin`

- **Nombre del contenedor:** `xuxemons-phpmyadmin`
- **Reinicio:** `always`
- **Variables de entorno:**
- `PMA\_HOST`: host de MySQL (`db`)
- `PMA\_PORT`: puerto MySQL (`3306`)
- `MYSQL\_ROOT\_PASSWORD`: contraseña root (`root`)
- **Puertos:** `8080:80` (accesible en `http://localhost:8080`)
- **Dependencia:** espera que el servicio `db` esté sano

12.3 Servicio `laravel`

- **Build:** construye la imagen desde `./docker/backend/laravel/Dockerfile`
- **Nombre del contenedor:** `xuxemons-laravel`
- **Reinicio:** `unless-stopped`
- **Directorio de trabajo:** `/var/www/html`
- **Variable de entorno:**
- `APACHE\_DOCUMENT\_ROOT`: `/var/www/html/public`
- **Volúmenes:**
- `./backend/laravel:/var/www/html:cached` (código fuente)
- `laravel\_vendor:/var/www/html/vendor` (dependencias persistentes)
- **Puertos:** `8000:80` (accesible en `http://localhost:8000`)
- **Comandos de inicialización:**

1. Copia `.env.docker` a `.env`
2. Instala dependencias de Composer
3. Espera a que MySQL esté listo
4. Limpia la base de datos (si existe)
5. Ejecuta migraciones
6. Ejecuta seeders
7. Limpia cache y config
8. Inicia Apache

12.4 Servicio `angular`

- **Imagen:** `node:20`
- **Nombre del contenedor:** `xuxemons-angular`
- **Reinicio:** `always`
- **Directorio de trabajo:** `/app`
- **Volúmenes:** `./frontend:/app` (código fuente)
- **Puertos:** `4200:4200` (accesible en `http://localhost:4200`)
- **Comando:** instala dependencias npm y ejecuta el servidor Angular
- **Dependencia:** espera a que `laravel` esté listo

12.5 Volúmenes

- `db\_data`: almacena los datos de MySQL de forma persistente
- `laravel\_vendor`: almacena las dependencias de Composer para no reinstalarlas siempre

13. Comandos básicos de uso

13.1 Iniciar los servicios

`docker compose up`

Esto inicia todos los servicios en modo foreground (mostrando logs en la terminal).

Alternativa (modo background):

`docker compose up -d`

El parámetro `-d` (detach) permite ejecutar los servicios en el fondo.

13.2 Ver estado de los servicios

`docker compose ps`

Muestra el estado de todos los contenedores, incluyendo si están ejecutándose y qué puertos exponen.

13.3 Construir las imágenes

`docker compose build`

Construye o reconstruye las imágenes (especialmente útil si cambian los Dockerfile).

13.4 Construir e iniciar (recomendado para primera vez)

`docker compose up --build`

Construye las imágenes nuevas y luego las inicia.

13.5 Detener los servicios

`docker compose stop`

Detiene los contenedores sin eliminarlos.

13.6 Reanudar servicios detenidos

`docker compose start`

Reanuda los contenedores que fueron detenidos.

13.7 Detener y eliminar todo

`docker compose down`

Detiene y elimina los contenedores (los volúmenes persistentes se mantienen).

Nota: para eliminar también los volúmenes:

`docker compose down -v`

13.8 Reconstruir todo desde cero

`docker compose down -v && docker compose up --build -d`

Elimina todo (incluyendo volúmenes), y luego construye e inicia nuevamente.

13.9 Ver logs

`docker compose logs -f`

Muestra logs en tiempo real. Añade `-f` para "follow" (seguir en vivo).

Logs de un servicio específico:

`docker compose logs -f laravel`

13.10 Ejecutar comandos en un contenedor en

ejecución `docker compose exec laravel php artisan migrate`

Ejecuta un comando en el contenedor Laravel sin necesidad de entrar en shell.

14. Pruebas de funcionamiento del despliegue

Las siguientes pruebas demuestran que el despliegue se ha completado exitosamente.

14.1 Versión de Docker Compose

Docker Compose version v2.40.3-desktop.1

14.2 Estado de contenedores

Resultado esperado después de ejecutar `docker compose ps`:

NAME	IMAGE	COMMAND	SERVICE
CREATED	STATUS	PORTS	
xuxemons-angular	node:20	"docker-entrypoint.s..."	angular
46 minutes ago	Up 4 seconds	0.0.0.0:4200->4200/tcp	
xuxemons-db	mysql:8.0	"docker-entrypoint.s..."	db
4 days ago	Up 46 minutes (healthy)	0.0.0.0:3306->3306/tcp	
xuxemons-laravel	aei-xuxemons-laravel	"docker-php-entrypoi..."	laravel
46 minutes ago	Up 45 minutes	0.0.0.0:8000->80/tcp	
xuxemons-phpmyadmin	phpmyadmin/phpmyadmin	"/docker-entrypoint...."	
phpmyadmin			
4 days ago	Up 46 minutes	0.0.0.0:8080->80/tcp	

Interpretación:

- Todos los contenedores están `Up` (en funcionamiento)
- El contenedor `db` muestra `(healthy)` confirmando que MySQL responde correctamente
- Los puertos están correctamente mapeados

14.3 Verificación de accesibilidad

Una vez desplegada la aplicación, es posible acceder a los siguientes puntos:

- **Frontend Angular:** `http://localhost:4200`
- **Backend API:** `http://localhost:8000/api`
- **phpMyAdmin:** `http://localhost:8080`
- **MySQL:** `localhost:3306` con usuario `xuxemons` / contraseña `xuxemons`

14.4 Criterios de despliegue exitoso

El despliegue se considera exitoso cuando:

1. Todos los contenedores están en estado `Up`.
2. El contenedor `db` muestra estado `(healthy)`.
3. El frontend responde en `http://localhost:4200`.
4. El backend responde en `http://localhost:8000/api`.
5. Los usuarios pueden registrarse y acceder a la aplicación.
6. Las operaciones CRUD en la base de datos funcionan correctamente.

15. Nota de implementación

Este documento describe el proyecto con el estado actual del código. Aunque el frontend se diseñó para ejecutarse en Docker, también es posible trabajar con él localmente mediante `npm install` y `npm start`. Debido a la configuración actual, el contenedor Angular puede requerir atención adicional con la instalación de dependencias si se ejecuta dentro de Docker.